# Informed Search - Part Two


**Arash Marioriyad** | **23 Farvardin 1404**

CE 417-Artificial Intelligence
Sharif Univ. of Tech.

Some of Slides have been adopted from Berkeley CS188

# Up To Now …

- Uninformed Search Algorithms (BSF, DFS, IDS, and UCS)
- Heuristics Function
- Informed Search
  - Greedy
  - A*

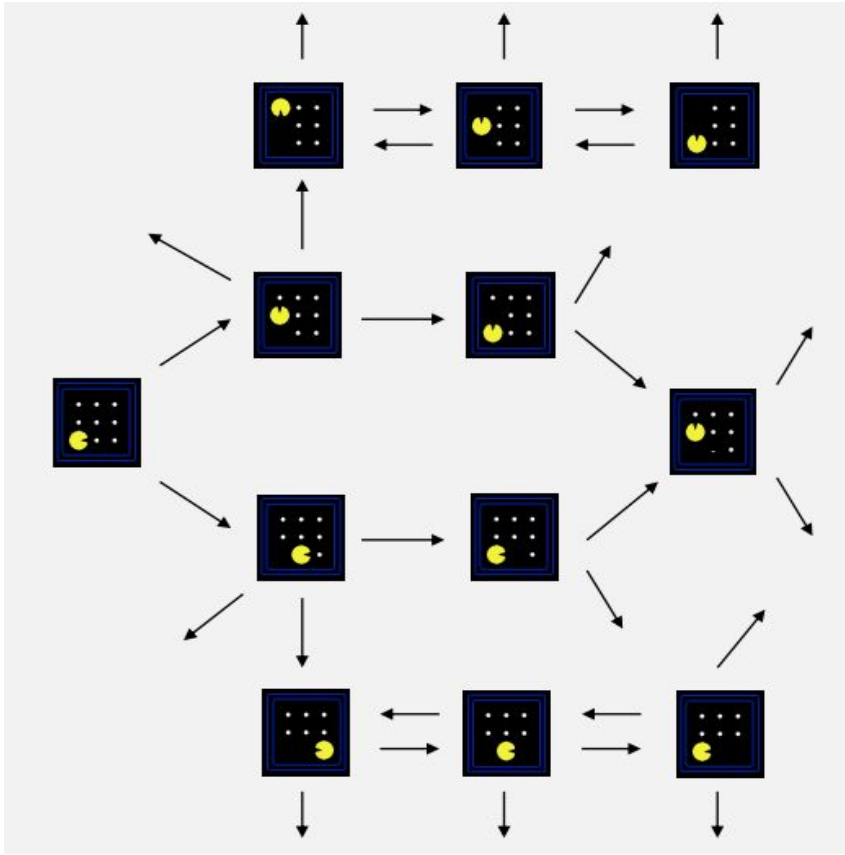
# Today: Continue to Informed Search

- Admissible Heuristics
- Graph Search
- Consistency of Heuristics

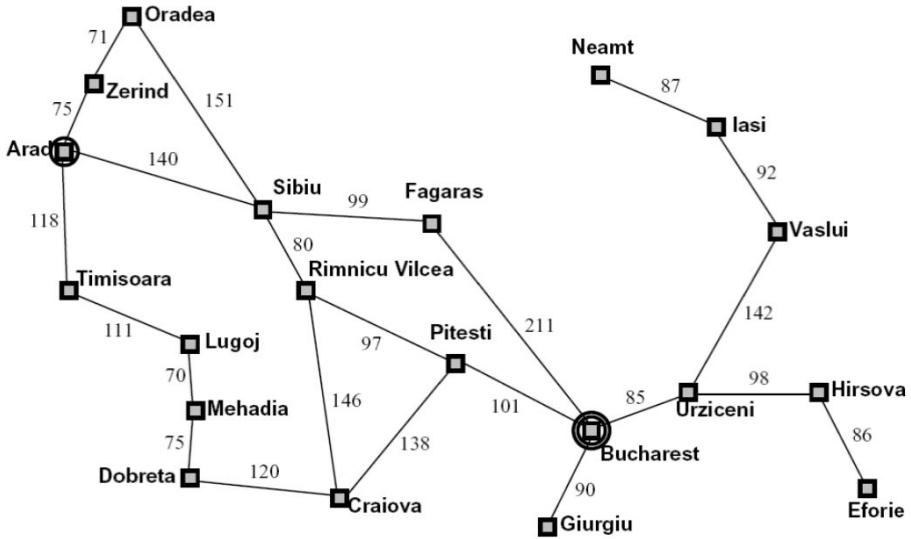


# Search Problem

- **States:** Configurations of the world
- **Actions**
- **Costs**
- **Start State**
- **Goal State**



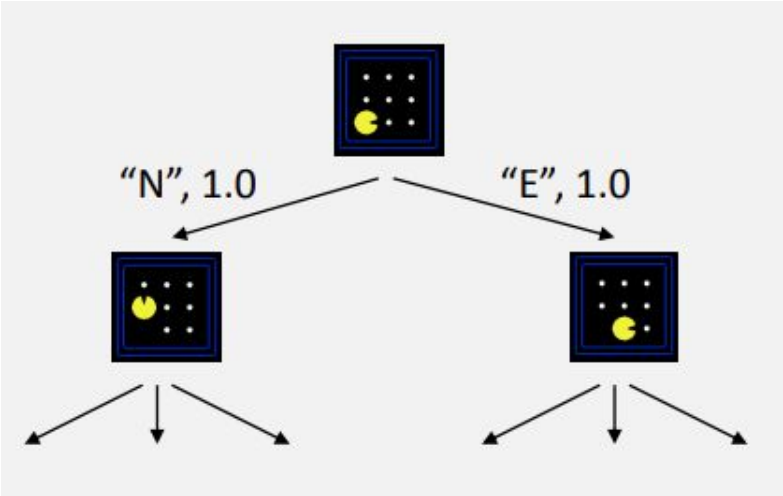
# Search Problem – Romania Map Problem

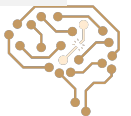



- State space:
  - Cities
- Successor function:
  - Roads: Go to adjacent city with cost = distance
- Start state:
  - Arad
- Goal test:
  - Is state == Bucharest?
- Solution?

# Search Tree

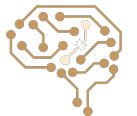
- A general framework for search algorithms
- Nodes are plans (sequence of states obtained by applying actions)
- Plans have costs
- We may see a state multiple times



# Tree Search

```
function TREE-SEARCH(problem, fringe) return a solution, or failure
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST(problem, STATE[node]) then return node
        for child-node in EXPAND(STATE[node], problem) do
            fringe ← INSERT(child-node, fringe)
        end
    end
```



# Search Algorithms

- **Complete:** It is guaranteed to find a solution whenever one exists.
- **Optimal:** It is guaranteed to find the best (lowest-cost) solution among all possible ones.
- **Time Complexity**
- **Space Complexity**



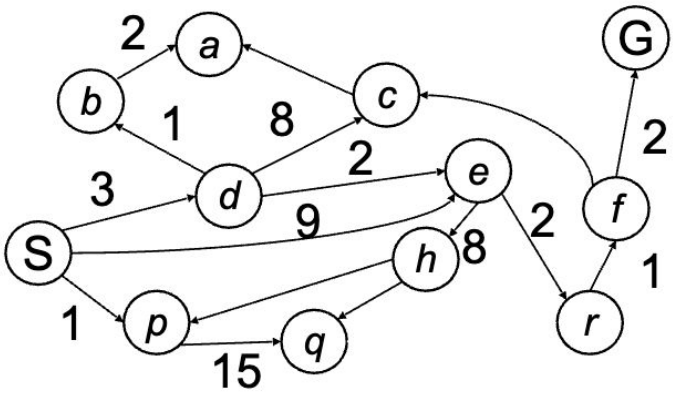
# Uninformed Search Algorithms

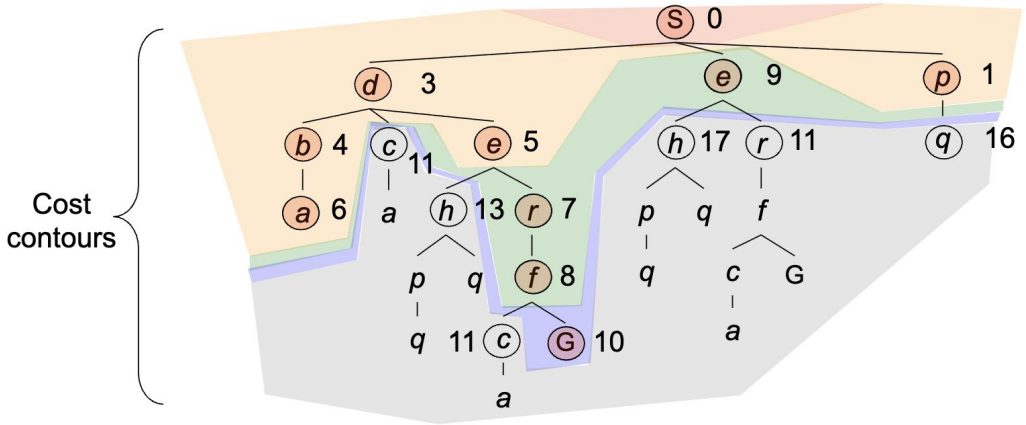
- **Breadth-First Search (BFS):**
  - Dequeues nodes in order of shallowest depth first (FIFO queue).
- **Depth-First Search (DFS):**
  - Dequeues the most recently added node first (LIFO stack).
- **Iterative Deepening Search (IDS):**
  - Repeatedly runs DFS, increasing the depth limit each time.
- **Uniform Cost Search (UCS):**
  - Dequeues the node with the lowest cumulative path cost (g(n)) first.



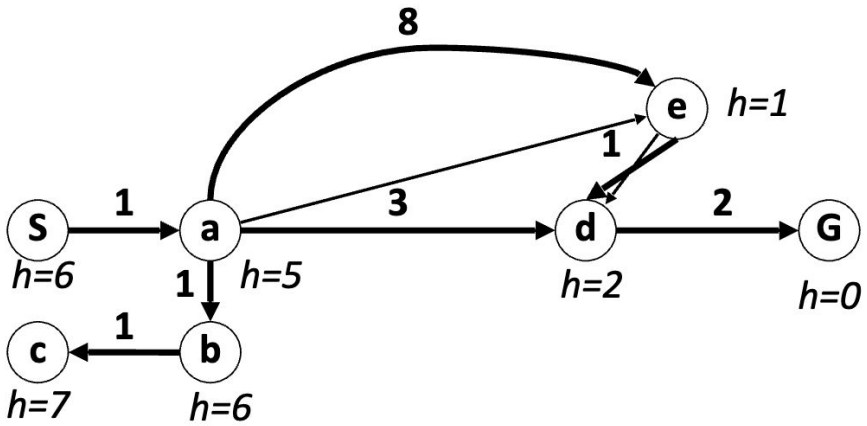
# UCS Algorithm

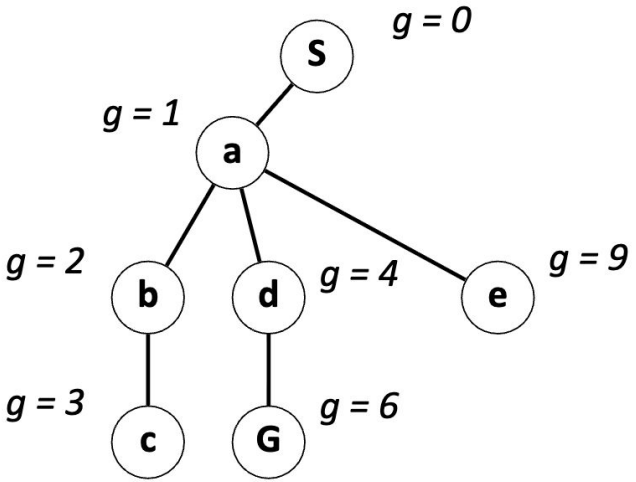
- g(n) = cost from root to n
- Strategy: expand lowest g(n)
- Frontier is a priority queue sorted by g(n)



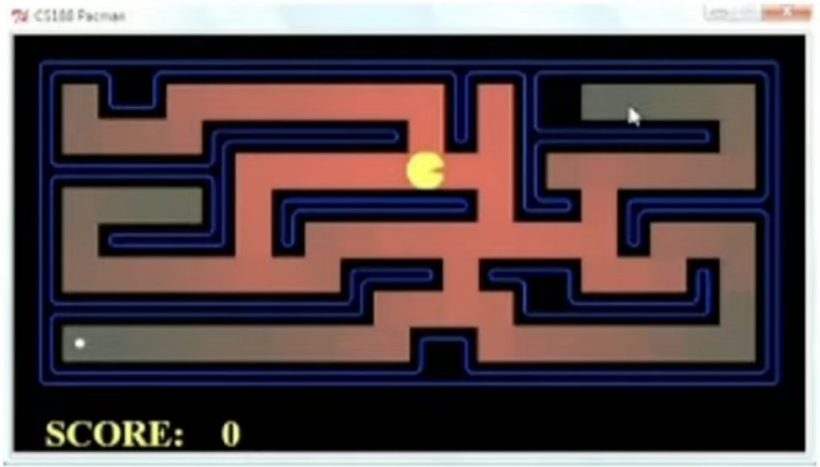
# UCS Example



# UCS Explores Everywhere!

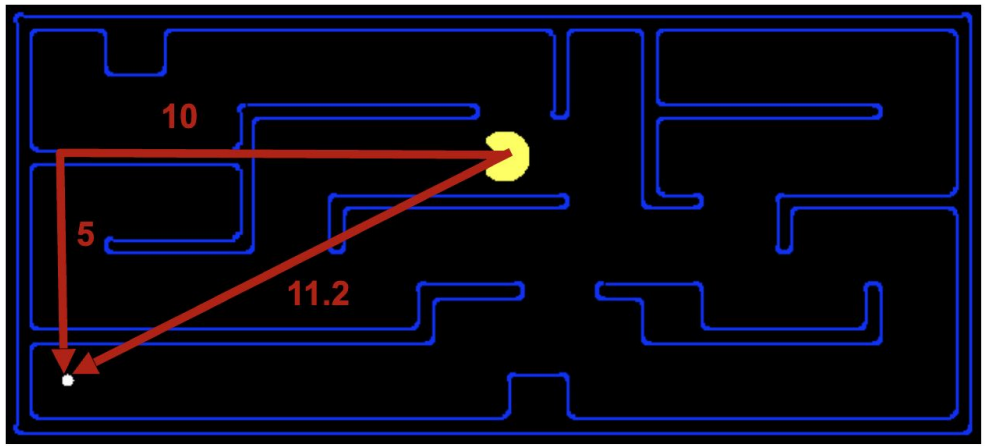
- UCS explores all plans with cost <k before explore a plan of cost k




Uniform-Cost Search

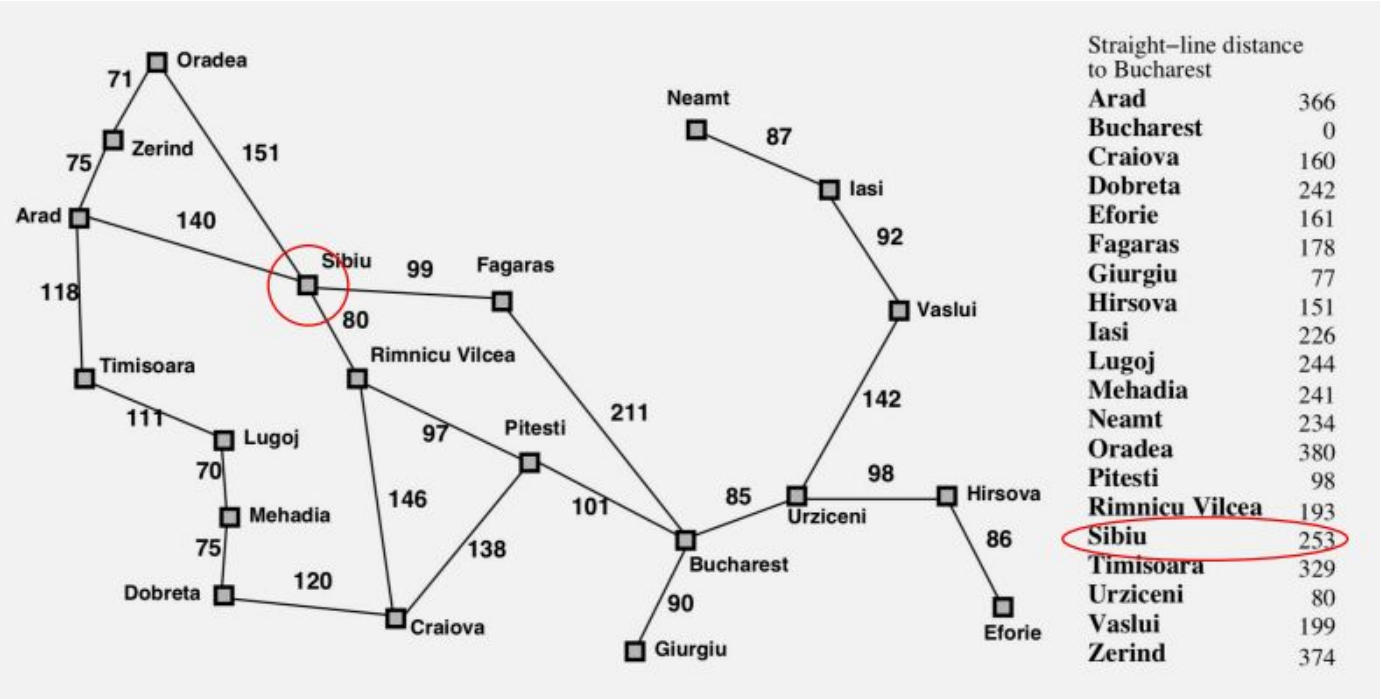
# Search Heuristics

- A function h(s): **estimates** how close a state is to a goal
- For each search problem we design/consider particular heuristics
- Example:
  - Manhattan distance
  - Euclidean distance



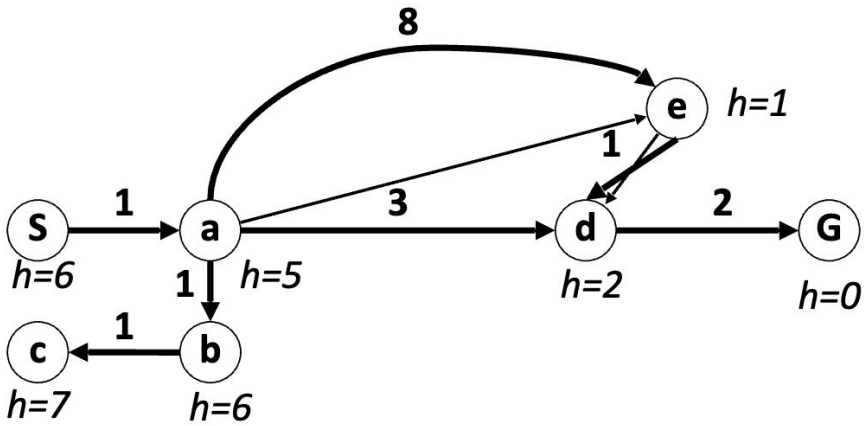
# State Space Graph – Romania Routing Problem

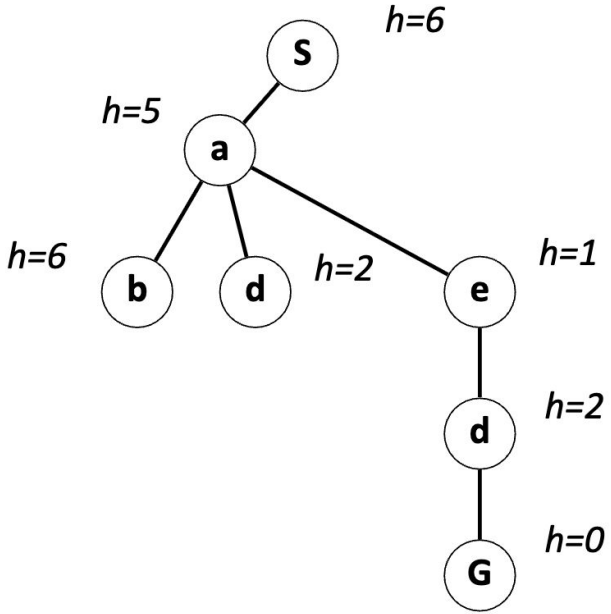
- Heuristic: Expand the node that seems closest (the lowest h(s))




| Straight–line distance to Bucharest | |
|---|---|
| Arad | 366 |
| Bucharest | 0 |
| Craiova | 160 |
| Dobreta | 242 |
| Eforie | 161 |
| Fagaras | 178 |
| Giurgiu | 77 |
| Hirsova | 151 |
| Iasi | 226 |
| Lugoj | 244 |
| Mehadia | 241 |
| Neamt | 234 |
| Oradea | 380 |
| Pitesti | 98 |
| Rimnicu Vilcea | 193 |
| Sibiu | 253 |
| Timisoara | 329 |
| Urziceni | 80 |
| Vaslui | 199 |
| Zerind | 374 |

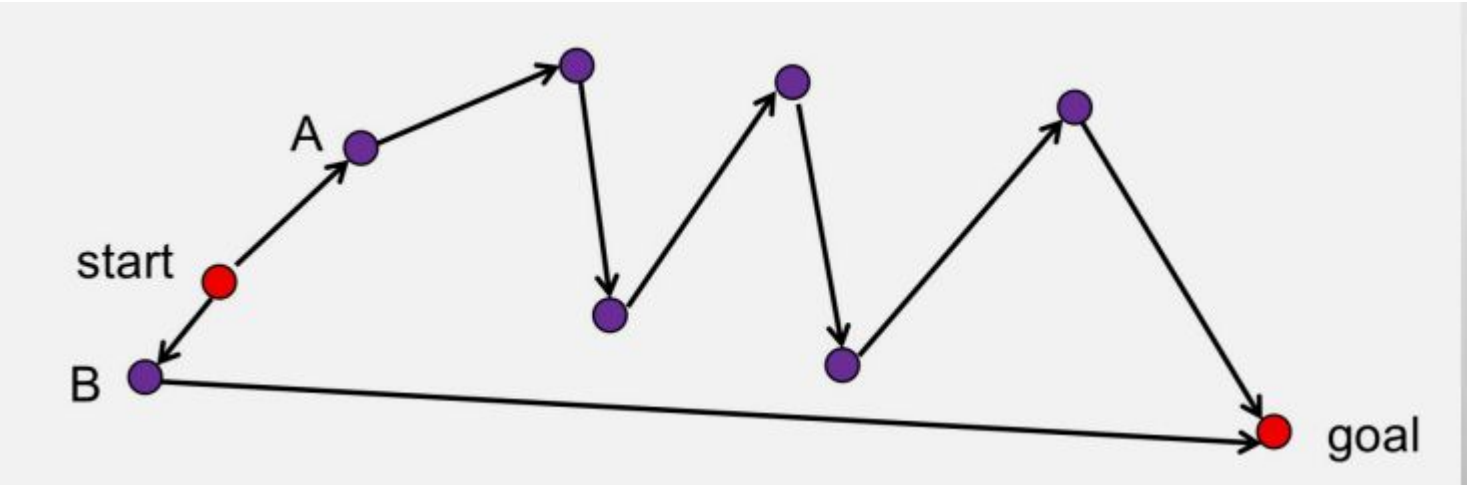
# Greedy Example



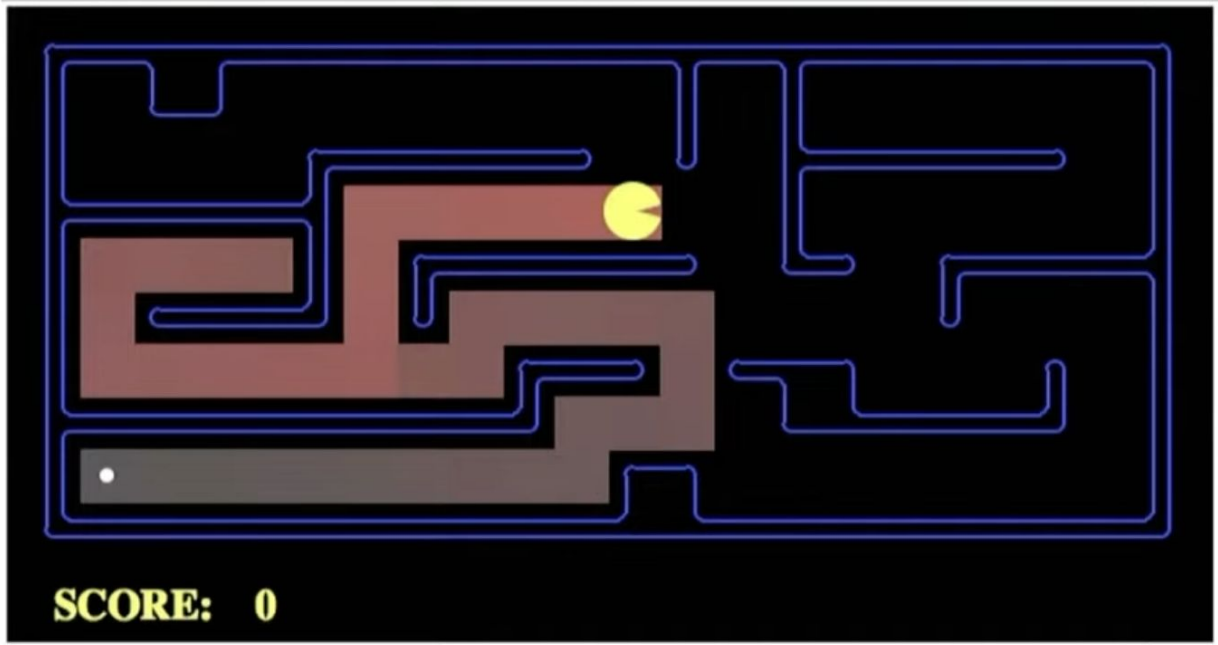



# How Can It Go Wrong?

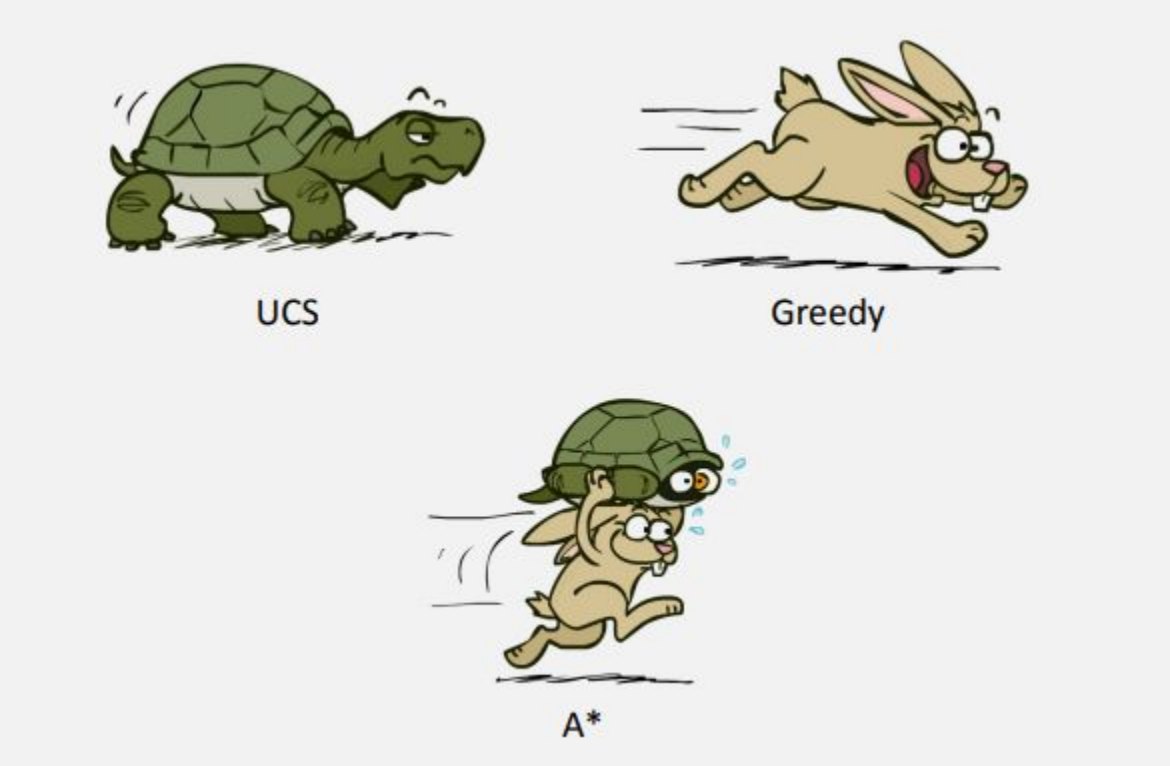
- Heuristic: Expand the node that seems closest...



# How Can It Go Wrong? – Pacman Small Maze

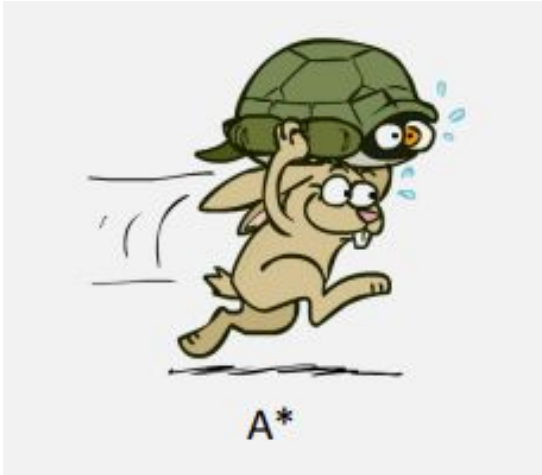


# Search Algorithms So Far

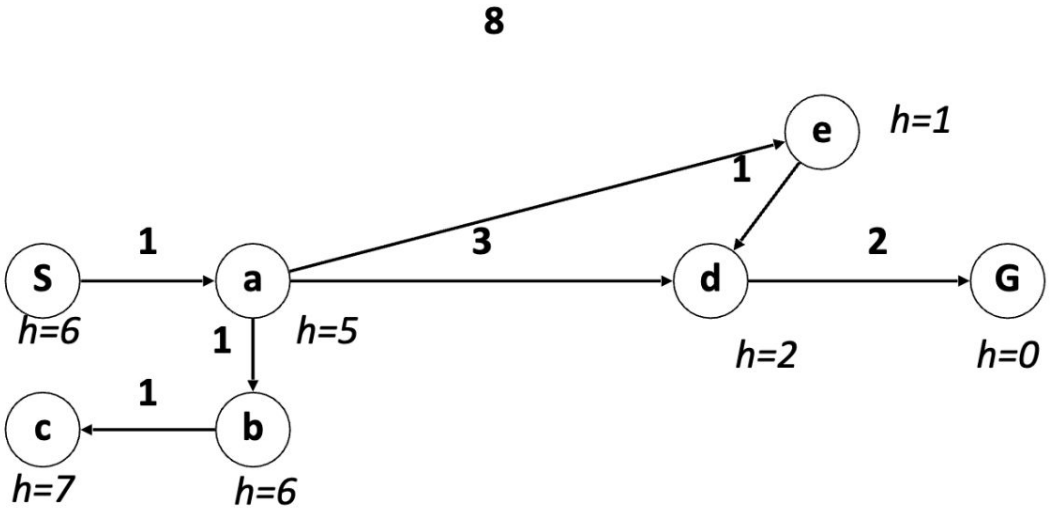


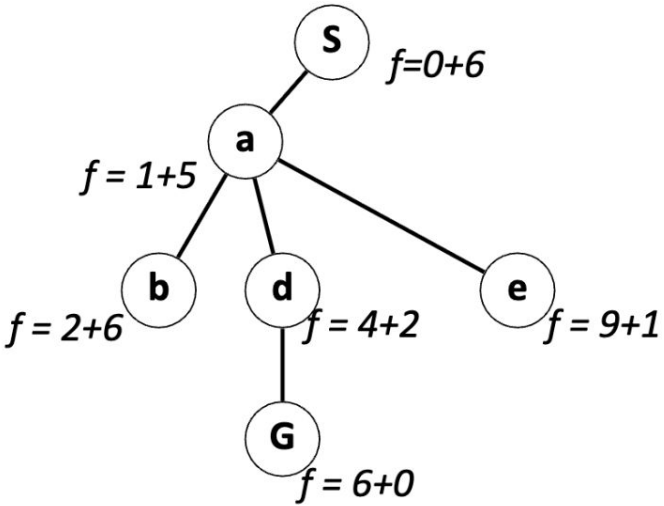
# A* Search – Combining UCS and Greedy

- Best first search with f(n) = g(n) + h(n)
- g(n) = sum of costs from start to n
- h(n) = estimate of the cost of path n → goal
- h(goal) = 0
- g(n) ~ uniform cost search
- h(n) ~ greedy search
- Best of both worlds ...

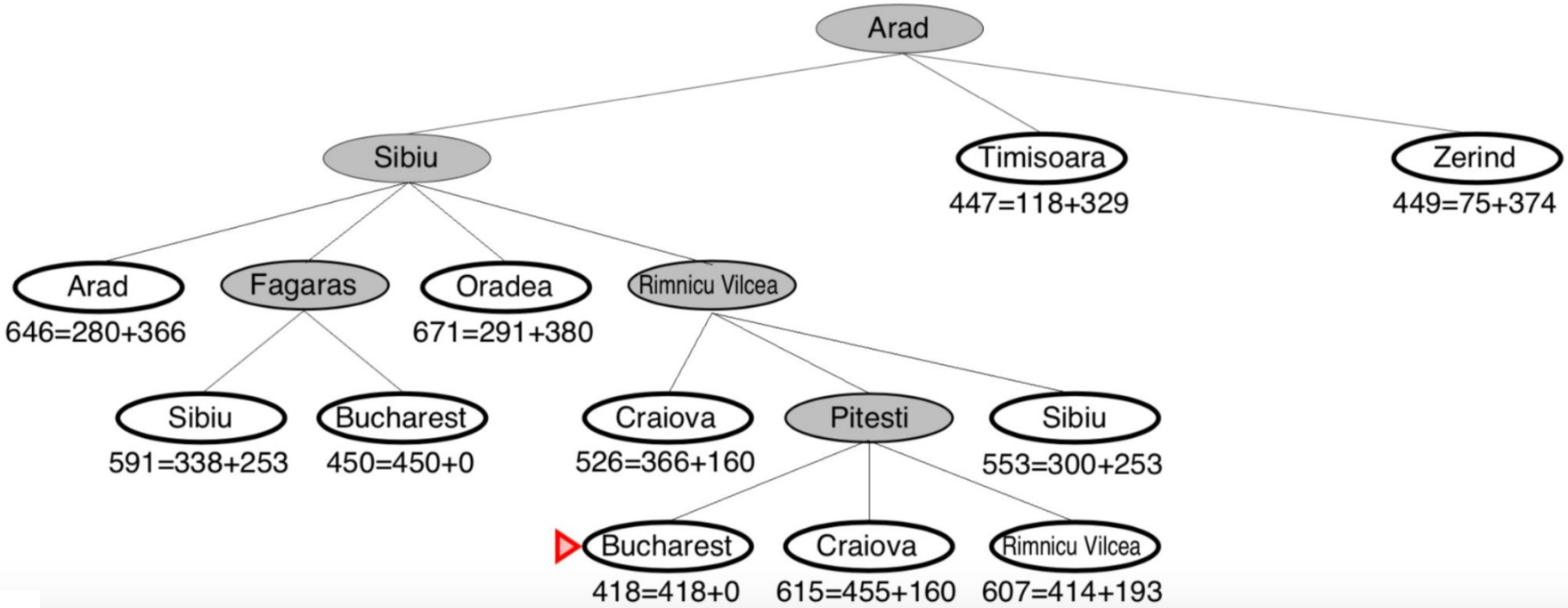



# A* Search – Example

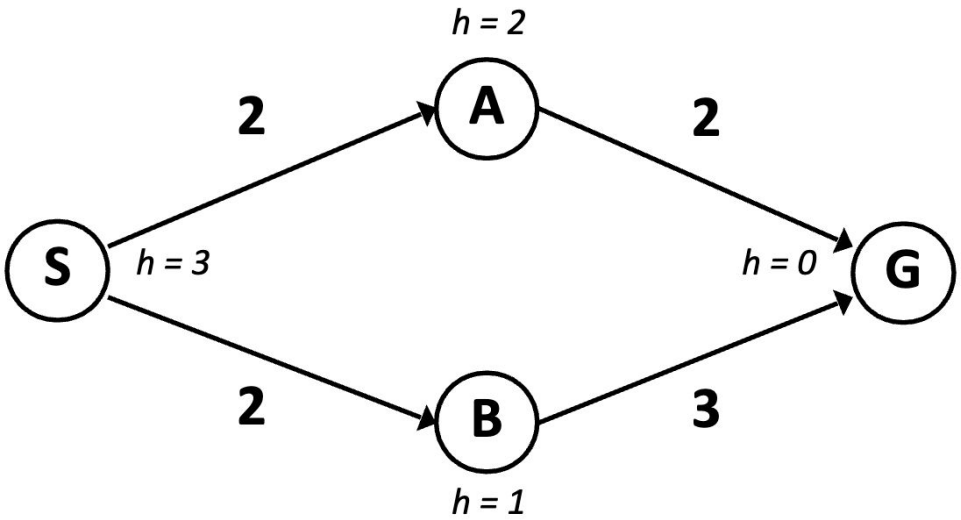


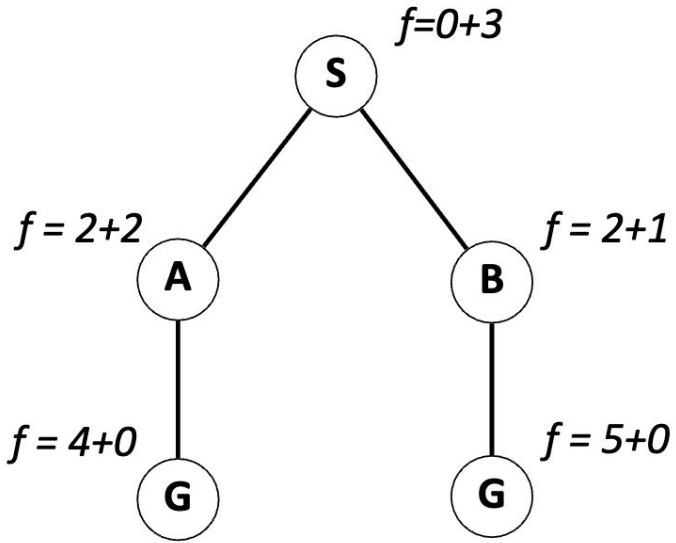



# A* Search – Romania Routing Example



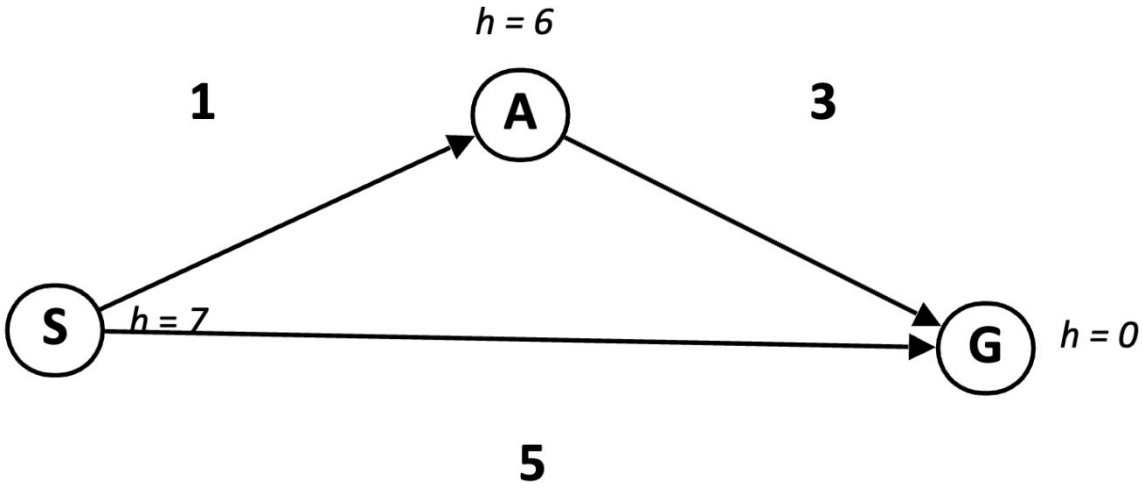
# When Should A* Search Terminate?

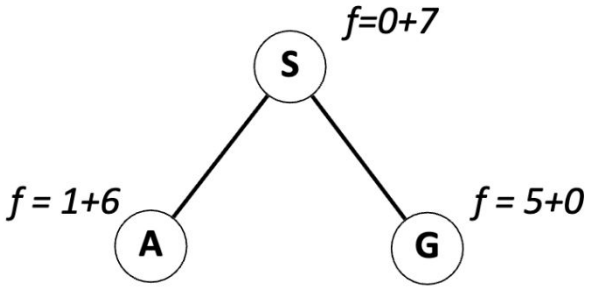



- Only stop when we dequeue a goal

# Is A* Search Optimal?

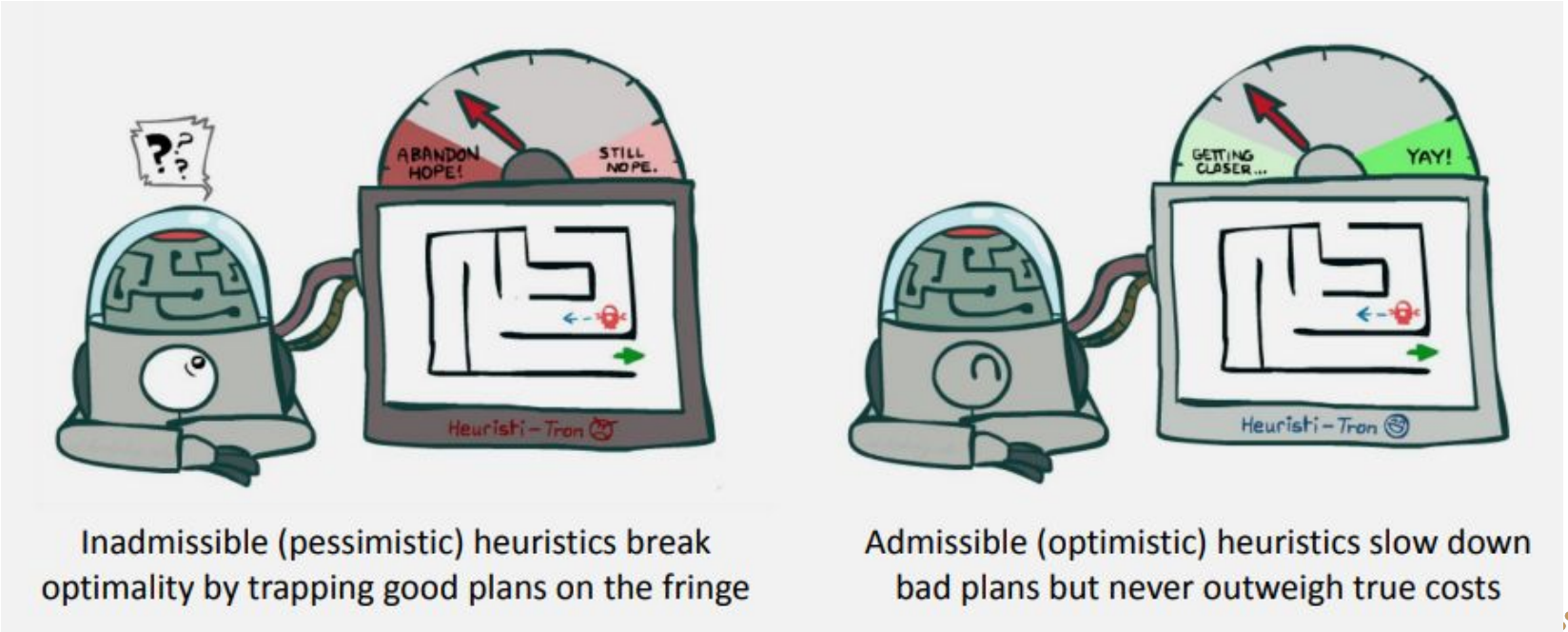



- What went wrong?
- Estimated good goal cost > Actual goal cost

# A* optimality (tree-search)?

- Theorem: If h(n) is admissible then A* is optimal in tree search




Inadmissible (pessimistic) heuristics break optimality by trapping good plans on the fringe

Admissible (optimistic) heuristics slow down bad plans but never outweigh true costs

# Admissible Heuristics

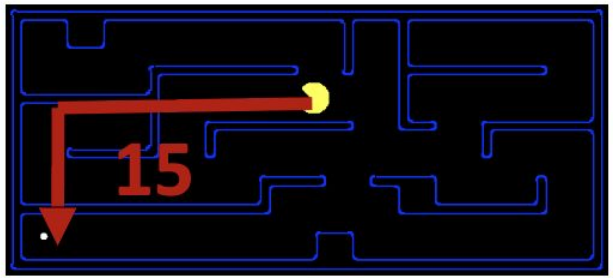
- A heuristic h is admissible (optimistic) if for every node n:

$$0 \leq h(n) \leq h^*(n)$$

- Where $h^*(n)$ is the true cost to a nearest goal
- Always underestimate the true cost to goal



# Admissible Heuristics – Examples

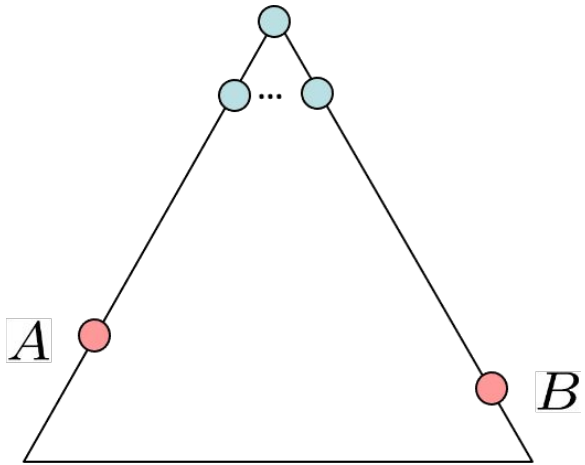



4

# A* Optimality in Tree Search?

- Assume:
  - A is an optimal goal node
  - B is a suboptimal goal node
  - h is admissible

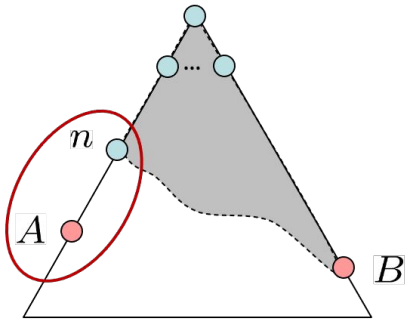
- Claim:
  - A will exit the fringe before B






# A* Optimality in Tree Search?

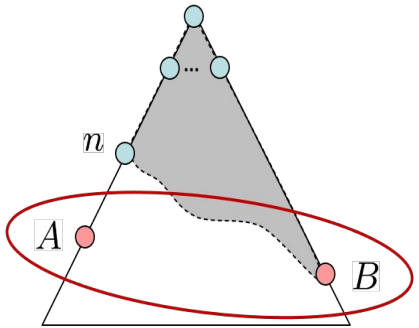
- Imagine B is on the fringe
- Some ancestor n of A is on the fringe,
- Maybe A itself is on the fringe too
- Claim: n will be expanded before B
  - f(n) is less or equal to f(A)




$$f(n) = g(n) + h(n) \quad \text{Definition of f-cost}$$
$$f(n) \leq g(A) \quad \text{Admissibility of h}$$
$$g(A) = f(A) \quad \text{h = 0 at a goal}$$

# A* Optimality in Tree Search?

- Imagine B is on the fringe
- Some ancestor n of A is on the fringe,
- Maybe A itself is on the fringe too
- Claim: n will be expanded before B
  - f(n) is less or equal to f(A)
  - f(A) is less or equal to f(B)

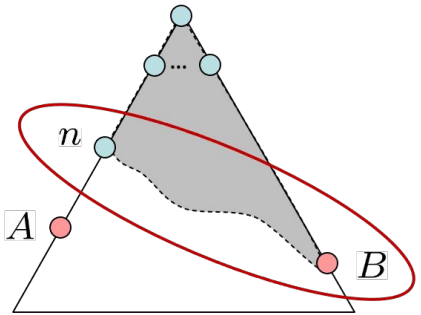



$g(A) < g(B)$ B is suboptimal

$f(A) < f(B)$ h = 0 at a goal

# A* Optimality in Tree Search?

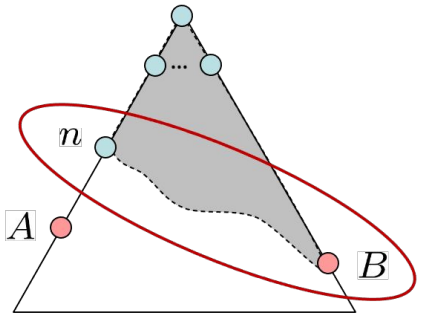
- Imagine B is on the fringe
- Some ancestor n of A is on the fringe,
- Maybe A itself is on the fringe too
- Claim: n will be expanded before B
  - f(n) is less or equal to f(A)
  - f(A) is less or equal to f(B)
  - n expands before B




$$f(n) \leq f(A) < f(B)$$

# Hence …

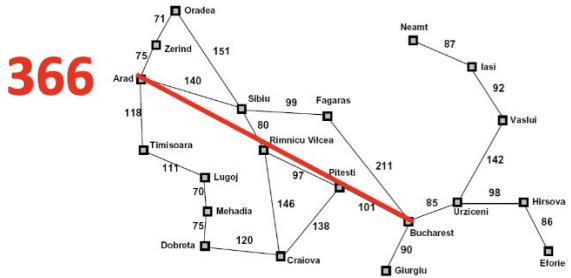
- All ancestors of A expand before B
- A expands before B
- A* search is optimal

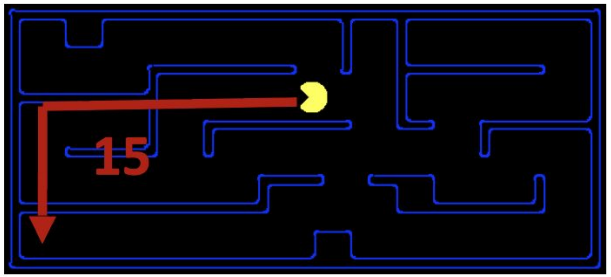



$$f(n) \leq f(A) < f(B)$$

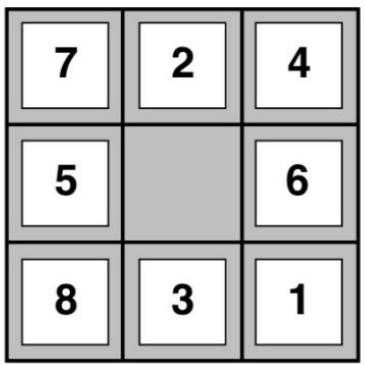
# Creating Admissible Heuristics

- Given a search problems, how to design an admissible heuristic?
- Often, admissible heuristics are solutions to relaxed problems
- Relaxed problem: Problem with fewer restrictions on the actions

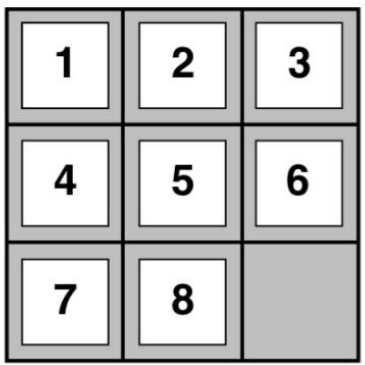


# Example: The 8-puzzle



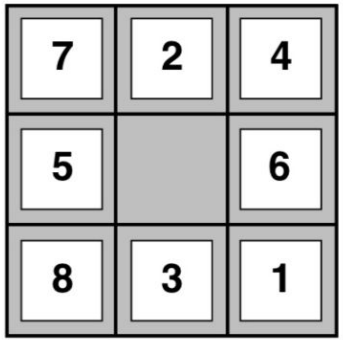

Start State



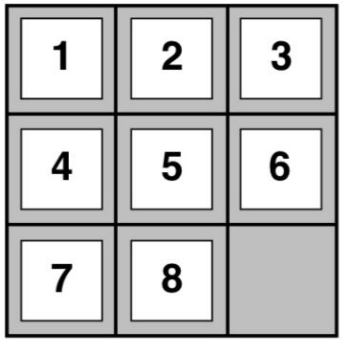

Goal State

# Example: The 8-puzzle

- $h_1(n)$ = Number of Misplaced Tiles → Admissible?
- $h_2(n)$ = Total Manhattan Distance → Admissible?



Start State



Goal State

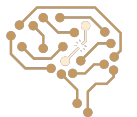
$h_1(S) =$?? 6

$h_2(S) =$?? 4+0+3+3+1+0+2+1 = 14

# Heuristics Dominance

- Suppose $h_1(n)$ and $h_2(n)$ are both admissible heuristics
- If $h_2(n) \geq h_1(n)$ for all n, then $h_2(n)$ dominates $h_1(n)$
- **Theorem:** if $h_2(n)$ dominates $h_1(n)$, $A^*$ with $h_2(n)$ expands at most the same number of nodes as when using $h_1(n)$, in search tree.
- Proof Hint: using expansion condition of $A^*$
  - $A^*$ expands a node n only if there exists a path through n such that: $f(n) <= C^*$





# Next Session: Graph Search & Local Search